

# SOC elemző rendszer tervezése Python nyelven

Kiber levelező 2026.04.11. jegyzet

Ez a jegyzet egy Windows-alapú SOC elemző rendszer felépítését és működését mutatja be tömören, lépésről lépésre. A Security Operations Center (SOC) rendszerek célja a szervezetek biztonságának folyamatos megfigyelése és a fenyegetések korai észlelése. A bemutatott rendszer egy egyszerűsített SOC megoldás, amely a naplóadatok gyűjtése és elemzése (SIEM) mellett helyszíni fájlintegritás-ellenőrzést (HIDS) is végez. Fő komponensei között szerepelnek a log-átalakító modul, egy anomália-detektor, egy integritás-ellenőrző és egy jelentéskészítő, amely HTML és JSON formátumban elkészíti az eredményeket. A rendszer moduláris felépítése lehetővé teszi a komponensek önálló tesztelését (unit tesztek) és a teljes együttműködés ellenőrzését integrációs tesztel. Az egységtesztekhez mesterséges, mock adathalmazokat használunk a reprodukálható támadási forgatókönyvek szimulálására, míg az integrációs teszt egy teljes end-to-end pipeline-t futtat le valódi logokkal.

A tanórai projekt ezen a fájlmegosztó linken található:

<https://git.nexttechnologies.hu/education/info2/-/tree/master/winsoc>

Ezen a linken belül találunk:

## doc:

- **WindowsSOCSecurityReport.png:** Így kellene kinéznie a jelentésnek☺
- **Követelmény specifikáció:** Így kezdünk egy projektet (Ady László készítette)
- **srs.tex:** ez is Követelmény specifikáció csak LaTeX formátumban!

**src:** (source) Itt található a teljes forráskód a **winsoc.py**-ban.

**test:** Itt található a teljes teszt kód **test\_winsoc.py**-ban

**.gitignore:** Azt mondja meg a Git-nek, hogy mely fájlokat és mappákat NE kövesse / NE tölts fel a repository-ba. Fő célja, hogy ne kerüljenek fel felesleges fájlok (pl. cache, build output) ne kerüljenek fel érzékeny adatok (pl. jelszavak, kulcsok) tiszta és kezelhető maradjon a projekt. **Ezzel nem kell foglalkozniuk nem a tananyag része!**

**README.md:** Windows SOC Analyst Tool (angol jegyzet)

**pyproject.toml:** Python projekt konfigurációs fájl, telepíthető SOC elemző eszköz, amelynek a neve: „winsoc”. Modulként alkalmazható és meghívható a `importal` a kódban vagy `pip`-ben telepíthető.

Kezdjük először a `src` fájl megnyitásával:

[https://git.nexttechnologies.hu/education/info2/-/tree/master/winsoc/src?ref\\_type=heads](https://git.nexttechnologies.hu/education/info2/-/tree/master/winsoc/src?ref_type=heads)

A kód felépítése, szerkezete:

1. TWIG-STYLE TEMPLATE ENGINE
2. MOCK LOG GENERATOR
3. WINDOWS LOG PARSER
4. ANOMALY DETECTOR
5. INTEGRITY CHECKER
6. REPORT GENERATOR
7. MAIN APPLICATION
8. COMMAND LINE INTERFACE

Használjunk fejlesztői környezetet az egyetem Pycharm-ot használ. Készítsünk egy új projektet legyen a neve WINSOC. Majd utána kezdjük egy új modulal (egységgel), amelynek a neve: Template\_engine.py ebbe másoljuk bele az src kód 1. részét:

```
# Template_engine.py
"""
Windows SOC Analyst Tool - Professional Cybersecurity Lab Exercise
Comprehensive incident response and log analysis tool with Twig-like templating
"""

# Röviden: ez a kód nem "nem csinál semmit", hanem csak egy építőelem
# konkrétan egy mini sablonmotor (template engine). Azért nem ír ki semmit,
# mert nincs benne olyan rész, ami futtatja vagy kiírja az eredményt.
# Mit csinál ez a kód?
# Ez a TemplateEngine osztály egy egyszerű HTML sablonfeldolgozó, hasonló a PHP-s Twig-hez.

import hashlib
import json
import re
import argparse
import datetime
import os
from collections import Counter
from pathlib import Path
from typing import Dict, List, Any, Optional, Tuple
import logging
import webbrowser

# ===== 1. TWIG-STYLE TEMPLATE ENGINE =====

class TemplateEngine:
    """
    Lightweight template engine supporting:
    - {{ variable }} variable substitution
    - {{ variable|length }} filters
    - {% for item in list %}...{% endfor %} loops
    - {% if condition %}...{% endif %} conditionals
    """

    def __init__(self, template_dir: str = "templates"):
        self.template_dir = Path(template_dir)
        self.template_dir.mkdir(exist_ok=True)

    def load_template(self, template_name: str) -> str:
        """Load template from file or create default"""
        template_path = self.template_dir / template_name
        if not template_path.exists():
            self.create_default_template(template_path)
```

```

with open(template_path, 'r', encoding='utf-8') as f:
    return f.read()

def _create_default_template(self, path: Path) -> None:
    """Create professional default HTML template"""
    html = """<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Windows SOC Security Report</title>
<style>
* { margin: 0; padding: 0; box-sizing: border-box; }
body {
    font-family: 'Segoe UI', 'Roboto', Arial, sans-serif;
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    padding: 20px;
    min-height: 100vh;
}
.container {
    max-width: 1400px;
    margin: 0 auto;
    background: white;
    border-radius: 15px;
    box-shadow: 0 20px 60px rgba(0,0,0,0.3);
    overflow: hidden;
}
.header {
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    color: white;
    padding: 30px;
    text-align: center;
}
.header h1 { font-size: 2.5em; margin-bottom: 10px; }
.header p { font-size: 1.1em; opacity: 0.9; }
.content { padding: 30px; }
.summary-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
    gap: 20px;
    margin-bottom: 30px;
}
.stat-card {
    background: #f8f9fa;
    padding: 25px;
    border-radius: 10px;
    text-align: center;
    transition: transform 0.2s;
    box-shadow: 0 2px 5px rgba(0,0,0,0.1);
}
.stat-card:hover { transform: translateY(-5px); }
.stat-card h3 { color: #667eea; margin-bottom: 10px; font-size: 1.2em; }
.stat-number { font-size: 3em; font-weight: bold; color: #333; }
.alert-box {
    background: #f8d7da;
    border-left: 5px solid #dc3545;
    padding: 20px;
    margin-bottom: 20px;
    border-radius: 8px;
}
.warning-box {
    background: #fff3cd;
    border-left: 5px solid #ffc107;
    padding: 20px;
    margin-bottom: 20px;
    border-radius: 8px;
}
.info-box {
    background: #d1ecf1;
    border-left: 5px solid #17a2b8;

```

```

padding: 20px;
margin-bottom: 20px;
border-radius: 8px;
}
table {
width: 100%;
border-collapse: collapse;
margin-top: 20px;
margin-bottom: 30px;
}
th, td {
border: 1px solid #ddd;
padding: 12px;
text-align: left;
}
th {
background: #667eea;
color: white;
font-weight: 600;
}
tr:nth-child(even) { background: #f8f9fa; }
tr:hover { background: #e9ecef; }
.badge {
display: inline-block;
padding: 4px 10px;
border-radius: 20px;
font-size: 0.85em;
font-weight: bold;
}
.badge-danger { background: #dc3545; color: white; }
.badge-warning { background: #ffc107; color: #333; }
.badge-success { background: #28a745; color: white; }
.footer {
background: #333;
color: white;
text-align: center;
padding: 20px;
font-size: 0.9em;
}
@media (max-width: 768px) {
.summary-grid { grid-template-columns: 1fr; }
th, td { font-size: 0.85em; padding: 8px; }
}
</style>
</head>
<body>
<div class="container">
<div class="header">
<h1>🛡️ Windows SOC Security Report</h1>
<p>Generated: {{ report_date }}</p>
<p>System: {{ system_name }}</p>
</div>

<div class="content">
<div class="summary-grid">
<div class="stat-card">
<h3>📊 Total Events</h3>
<div class="stat-number">{{ total_events }}</div>
</div>
<div class="stat-card">
<h3>⚠️ Suspicious IPs</h3>
<div class="stat-number">{{ suspicious_ips|length }}</div>
</div>
<div class="stat-card">
<h3>🔒 Integrity Changes</h3>
<div class="stat-number">{{ integrity_changes|length }}</div>
</div>
<div class="stat-card">
<h3>🚨 Alerts</h3>
<div class="stat-number">{{ alerts|length }}</div>

```

```

</div>
</div>

{% if suspicious_ips %}
<div class="warning-box">
  <h2><img alt="Warning icon" data-bbox="178 108 198 123"/> Suspicious IP Addresses Detected</h2>
  <p>The following IP addresses showed malicious activity:</p>
  <ul>
    {% for ip in suspicious_ips %}
    <li><strong>{{ ip }}</strong></li>
    {% endfor %}
  </ul>
</div>
{% endif %}

{% if brute_force_ips %}
<div class="alert-box">
  <h2><img alt="Alert icon" data-bbox="178 265 198 280"/> Brute Force Attack Detected</h2>
  <p>Multiple failed login attempts from:</p>
  <ul>
    {% for ip in brute_force_ips %}
    <li><strong>{{ ip }}</strong> - {{ brute_force_counts[ip] }} failed attempts</li>
    {% endfor %}
  </ul>
</div>
{% endif %}

{% if integrity_changes %}
<div class="warning-box">
  <h2><img alt="Warning icon" data-bbox="178 423 198 438"/> File Integrity Changes</h2>
  <table>
    <thead>
      <tr><th>File Path</th><th>Status</th></tr>
    </thead>
    <tbody>
      {% for change in integrity_changes %}
      <tr>
        <td>{{ change.file }}</td>
        <td><span class="badge badge-danger">{{ change.status }}</span></td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
</div>
{% endif %}

{% if alerts %}
<div class="alert-box">
  <h2><img alt="Alert icon" data-bbox="178 671 198 686"/> Alert List</h2>
  {% for alert in alerts %}
  <p><strong><img alt="Warning icon" data-bbox="208 701 228 716"/></strong> {{ alert }}</p>
  {% endfor %}
</div>
{% endif %}

{% if events %}
<h2><img alt="Event log icon" data-bbox="178 778 198 793"/> Detailed Event Log</h2>
<table>
  <thead>
    <tr><th>Timestamp</th><th>Event Type</th><th>Source IP</th><th>Details</th></tr>
  </thead>
  <tbody>
    {% for event in events %}
    <tr>
      <td>{{ event.timestamp }}</td>
      <td>{{ event.type }}</td>
      <td>{{ event.ip }}</td>
      <td>{{ event.details }}</td>
    </tr>
    {% endfor %}
  </tbody>
</table>

```

```

        {% endfor %}
    </tbody>
</table>
{%% endif %}
</div>

<div class="footer">
    <p>Windows SOC Analyst Tool - Cybersecurity Lab Exercise</p>
    <p>This report was automatically generated by the incident response system</p>
</div>
</div>
</body>
</html>""""
    with open(path, 'w', encoding='utf-8') as f:
        f.write(html)

```

```
def render(self, template: str, context: Dict[str, Any]) -> str:
```

```
    """Render template with given context"""
```

```
    result = template
```

```
    # Handle {% if condition %}...{% endif %}
```

```
    if_pattern = r'\{% if ([\w\.]*) %\}(.*)%\{% endif %\}'
```

```
def replace_if(match):
```

```
    condition = match.group(1)
```

```
    content = match.group(2)
```

```
    value = self._get_nested(context, condition)
```

```
    if value:
```

```
        return self.render(content, context)
```

```
    return "
```

```
result = re.sub(if_pattern, replace_if, result, flags=re.DOTALL)
```

```
    # Handle {% for item in list %}...{% endfor %}
```

```
    for_pattern = r'\{% for (\w+) in ([\w\.]*) %\}(.*)%\{% endfor %\}'
```

```
def replace_for(match):
```

```
    var_name = match.group(1)
```

```
    list_path = match.group(2)
```

```
    content = match.group(3)
```

```
    items = self._get_nested(context, list_path)
```

```
    if not isinstance(items, list):
```

```
        return "
```

```
    rendered_items = []
```

```
    for item in items:
```

```
        new_context = context.copy()
```

```
        new_context[var_name] = item
```

```
        if isinstance(item, dict):
```

```
            for k, v in item.items():
```

```
                new_context[f'{var_name}.{k}'] = v
```

```
        rendered_items.append(self.render(content, new_context))
```

```
    return ".join(rendered_items)
```

```
result = re.sub(for_pattern, replace_for, result, flags=re.DOTALL)
```

```
    # Handle {{ variable }} and {{ variable|filter }}
```

```
    var_pattern = r'\{\{s*([\w\.]*)?(?:\((\w+)\)?s*\}\}'
```

```
def replace_var(match):
```

```
    var_path = match.group(1)
```

```
    filter_name = match.group(2)
```

```
    value = self._get_nested(context, var_path)
```

```
    # Apply filters
```

```
    if filter_name == 'length':
```

```
        return str(len(value)) if hasattr(value, '__len__') else '0'
```

```
    elif filter_name == 'upper':
```

```
        return str(value).upper() if value else "
```

```
    elif filter_name == 'lower':
```

```

        return str(value).lower() if value else "
    elif filter_name == 'default':
        return str(value) if value else "

    return str(value) if value is not None else "

result = re.sub(var_pattern, replace_var, result)

return result

def get_nested(self, context: Dict, path: str) -> Any:
    """Get nested dictionary value using dot notation (e.g., 'user.name')"""
    parts = path.split('.')
    value = context
    for part in parts:
        if isinstance(value, dict):
            value = value.get(part)
        elif hasattr(value, part):
            value = getattr(value, part)
        else:
            return None
    return value

def render_template(self, template_name: str, context: Dict[str, Any]) -> str:
    """Load and render a template"""
    template_content = self.load_template(template_name)
    return self.render(template_content, context)

engine = TemplateEngine()

context = {
    "report_date": "2026-04-18",
    "system_name": "My PC",
    "total_events": 10,
    "suspicious_ips": ["192.168.1.1"],
    "integrity_changes": [],
    "alerts": ["Brute force detected"],
    "events": []
}

html = engine.render_template("report.html", context)

filename = "report.html"

with open(filename, "w", encoding="utf-8") as f:
    f.write(html)

path = os.path.abspath(filename)
webbrowser.open(f"file://{path}")

```

## Template Engine

A jelentéskészítéshez a rendszer egy Twig-stílusú sablonmotort használ. Ez a motor lehetővé teszi, hogy a HTML riportba beágyazzunk változókat, ciklusokat és logikai utasításokat. Például a sablonban `{{ variable }}` szintaxis helyettesít egy változó értékét, és `{% for ... %}` ciklusszintaxis iterál egy listán. Így dinamikusan generálhatók a dashboard elemek és táblázatok a végrehajtott elemzések alapján.

A sablonmotor segítségével HTML-jelentések generálhatók dinamikusan változó adatokból (változók, ciklusok, feltételek)

Fontos hogy megjelenítéskor másolják be az utolsó kis kódrészletet (**ez nincs az eredeti src kódban!!!**: továbbá a legelejére ezt a modult: `import webbrowser`

```

engine = TemplateEngine()

context = {
    "report_date": "2026-04-18",
    "system_name": "My PC",
    "total_events": 10,
    "suspicious_ips": ["192.168.1.1"],
    "integrity_changes": [],
    "alerts": ["Brute force detected"],
    "events": []
}

html = engine.render_template("report.html", context)

filename = "report.html"

with open(filename, "w", encoding="utf-8") as f:
    f.write(html)

path = os.path.abspath(filename)
webbrowser.open(f"file://{path}")

```

## Mock Log generator

Szimulálja a különféle belépéseket a rendszerbe tanulás céljából, hogy adataink legyenek, amelyekkel később dolgozni tudunk.

3 darab teszt log fájlt generál (gyökérben látható lesz, amelyeket meg is nyithatnak):

- security\_test.log (biztonsági bejelentkezéseket készít, sikeres v sikertelent, mint ha valaki próbálgatná a jelszavakat brute force támadásnál)
- powershell\_test.log (imitálja pl.: malware letöltést, új felhasználó létrehozását (backdoor)stb.)
- iis\_test.log (webszerver bejelentkezések szimulálása pl.: normál kérés -> GET / index.html 200)

Ez a három együtt egy mini támadási forgatókönyv tesztelés céljából.

A generátor futtatása ezt kell a kód végére írni!!:

```

if __name__ == "__main__":
    path = MockLogGenerator.generate()
    print('ide tette őket: ', path)

```

## WindowsLogParser:

Bejelentkezések feldolgozását látja el, ahol kimenetként megjeleníti a sikeres vagy sikertelen bejelentkezést. Nyilván attól függ, hogy melyik generált logot kell kiértékelni(lásd Mock Log Generátor részt)!

**Futtatáshoz és megjelenítéshez:**

1. security\_test.log esetében ezt kell beírni:



```
parser = WindowsLogParser('test_logs/security_test.log')
events = parser.parse_security_log()
for log in events:
    print(log)
```

## 2. powershell\_test.log-nél pedig ezt:

```
parser = WindowsLogParser('test_logs/powershell_test.log')
events = parser.parse_powershell_log()
for log in events:
    print(log)
```

## 3. iis\_test.log kiírásánál:

```
parser = WindowsLogParser('test_logs/iis_test.log')
events = parser.parse_iis_log()
for log in events:
    print(log)
```

## Anomaly Detector

Mintát keres a már feldolgozott adatokban. Anomáliákat elemez a korábban már generált bejelentésekből. Ennek megfelelően támadásokat ismer fel, gyanús viselkedéseket szűri ki és riasztásokat generál.

**detect\_brute\_force()** elemzi, hogy hány sikertelen bejelentkezés van a generált logokban. Esetünkben meg van adva, hogy 3-nál több van, akkor riasztás.

**detect\_port\_scan()** port scan támadások kiszűrésére

**detect\_malicious\_powershell()** kiszűri a gyanús powershell eseményeket

**generate\_alerts()** riasztásokat megjeleníti.

## Futtatás és megjelenítés:

```
# 1. parser-be konvertáljuk a logot:
parser = WindowsLogParser("test_logs/security_test.log")
security_events = parser.parse_security_log()

# 2. detector
detector = AnomalyDetector(brute_force_threshold=3)

# 3. brute force keresés
suspicious_ips = detector.detect_brute_force(security_events)

# 4. alert generálás
alerts = detector.generate_alerts(suspicious_ips)

# 5. kiírás
for alert in alerts:
    print(alert)
```

## Integrity checker

Rendszerfájlok manipulálásának figyelésére használjuk. Fájlok változását figyeli, úgy hogy először eltárolja hash formátumban, majd később ellenőrzi hogy van-e bármiféle változás a hash-ben.

Lépései a kódnak:

**create\_baseline()** minden generált lognál(security\_test.log, powershell\_test.log és iis\_test.log) készít egy-egy Hash kódot majd eltárolja egy baseline.json fájlba.

**verify\_integrity()** összehasonlítja az aktuális fájlt hash-ét a baseline.json hash-ével

**calculate\_hash()** SHA-256 kiszámítása

**get\_windows\_critical\_paths()** előre megírt kritikus útvonalak

### futtatás és megjelenítés:

```
checker = IntegrityChecker()
checker.create_baseline('test_logs')
print('baseline kész')
change = checker.verify_integrity()
for file, status in changes:
    print(file, status)
```

## Report generator

Elemzésekből kimutatást készít két formában: HTML és JSON

kiíratáshoz kell egy Template engine

## Main application

Teljes rendszer vezérlését végzi el. Az egész SOC projekt összehangolását és futtatását végzi. Minden eddigi modult feldolgozz. Inicializál, a generált logokat beolvassa, átalakítja elemez és végül futtatja. Egész kódsorozat (amit eddig vettünk) koordinálásáért és futtatásáért felelős kódsor. Tehát a 3 darab generált logot beolvassa, feldolgozza(WindowsLogParser), Ennek alapján támadásokat és egyéb anomáliákat ellenőriz és felismeri. Majd a generált logokat hash-eli és ellenőrzi épségüket és végül egy webes kimutatást készít az eredményekből.

Ezt kell még beírni, ha futtatni akarjuk:

Fontos a kód elejére importáljuk az összes eddigi kódrészletet:

**from** Template\_engine **import** TemplateEngine....stb:

```
# main_ application.py
# -----MAIN Application -----
from Template_engine import TemplateEngine
from Log_generator import MockLogGenerator
from Win_Log_parsel import WindowsLogParser
from Anomaly_Detector import AnomalyDetector
from Integrity_checker import IntegrityChecker
from report_generator import ReportGenerator

import datetime
import os
from pathlib import Path
from typing import Dict, List, Any, Optional, Tuple
import logging

.
.
.
.
```

És a kód végére a megjelenítés miatt:

```
analyzer = WindowsSOCAnalyzer()

results = analyzer.run\_full\_analysis(
    security_log="test_logs/security_test.log",
    powershell_log="test_logs/powershell_test.log",
    iis_log="test_logs/iis_test.log",
    check_integrity=False
)

print(results)
```

## Command line interface

A parancssor vezérlőfelülete, így egyetlen paranccsal tudjuk futtatni a program teljes egészét! Feldolgozza a parancsolat és eldönti, hogy mit kell futtatni, majd meghívja a Main application modult.

Fontos a kód elejére importáljuk az összes eddigi kódrészletet:

```
# Command_line interface.py
# ===== 8. COMMAND LINE INTERFACE =====
from Template_engine import TemplateEngine
from Log_generator import MockLogGenerator
from Win_Log_parsel import WindowsLogParser
from Anomaly_Detector import AnomalyDetector
from Integrity_checker import IntegrityChecker
from report_generator import ReportGenerator
from main_application import WindowsSOCAnalyzer

import argparse
import datetime
import os
from pathlib import Path
from typing import Dict, List, Any, Optional, Tuple
import logging
```

## Tesztelés

Automatikus tesztcsomag a célja, hogy a modulok külön-külön is helyesen működnek-e. Tehát a logika valóban azt csinálja, amit kell!

Teszteli a **Template engine-t**, úgy hogy behelyettesíti a változókat, ellenőrzi a ciklusokat, szűrőket és a fontosabb objektumokat.

**Mock Log generátornál:** teszteli, hogy létre jötte-e a generált 3 fájl, tartalmazzák-e a megfelelő mintázatot stb.

**Win Log Parsel:** helyesen dolgozta fel a generált bejelentkezéseket fájlokat?

**Anomália detektor:** A beírt eseményeket – mint a generált logoknál – teszteli, hogy a program logikája szerint hajtódik-e végre.

**Integrity checker:** a hash-elés és a baseline fájl létrejött?, az esetleges módosítást felismeri? stb.

**Report generátor:** létrejött a két fájl formátum? Az adatok helyesek és meg is jelennek a kimutatásban?

Végül az „egész programcsomag működik” tesztel zárul a kód.

**Zárszó:** A teljes kód beillesztésével is lehet futtatni a forráskódot és tesztelést is. A részekre való bontás oktatás céljából van, amely egy moduláris programozási gyakorlatot

tesz lehetővé. Ha például több 1000 sorból állna egy program célszerű lenne blokkokra osztani az átláthatóság és a fejlesztés hatékonysága szempontjából. Egy fejlesztő csapatnál minden egyes fejlesztőnek kidelegálják a feladatokat, hogy mely kódrészlettel kell foglalkozni és ezeket majd össze kell rakni egy egységgé!